

Rapport final

Federated Learning appliqué à la classification d'images

Implémentation from scratch, simulation distribuée et Substra

Cours : 8INF887 – Apprentissage profond

Étudiant : Vinith Naresh Prabakaran

Date : 4 mai 2026

Table des matières

1	Introduction	2
2	Principe du Federated Learning	2
3	Implémentation from scratch	2
3.1	Baseline centralisée	2
3.2	Première version fédérée	3
4	Étude des distributions IID et non-IID	3
5	Comparaison des modèles	3
5.1	MLP	3
5.2	CNN V1	3
5.3	CNN V2	4
5.4	CNN V3	4
6	Exploration avec autoencodeur	4
7	Simulation distribuée avec Streamlit	5
8	Intégration Substra	6
9	Discussion	7
10	Limites et perspectives	8
11	Conclusion	8

1 Introduction

Ce projet porte sur l'apprentissage fédéré, ou *Federated Learning*, appliqué à la classification d'images avec le dataset MNIST.

L'objectif était de comprendre comment entraîner un modèle global sans centraliser les données. Ce type d'approche est particulièrement utile dans des contextes où les données sont sensibles, comme le domaine médical, où plusieurs institutions peuvent collaborer sans partager directement leurs données.

Le projet a été réalisé progressivement :

- une implémentation from scratch de FedAvg avec PyTorch ;
- une étude des distributions IID et non-IID ;
- une comparaison de plusieurs modèles, dont MLP et CNN ;
- une interface Streamlit simulant plusieurs clients et un serveur ;
- une intégration de Substra pour se rapprocher d'un cadre industriel.

Le but du projet n'était pas seulement d'obtenir une bonne accuracy sur MNIST, mais surtout de comprendre les contraintes réelles du Federated Learning : distribution des données, agrégation, rounds, choix du modèle et évaluation.

2 Principe du Federated Learning

En apprentissage fédéré, les données restent sur les clients. Le serveur central ne reçoit pas les données brutes, mais uniquement les poids ou les mises à jour des modèles locaux.

Le processus général est le suivant :

1. le serveur initialise un modèle global ;
2. chaque client reçoit ce modèle ;
3. chaque client entraîne localement sur ses propres données ;
4. les modèles locaux sont envoyés au serveur ;
5. le serveur agrège les modèles pour produire un nouveau modèle global.

L'algorithme utilisé dans ce projet est **FedAvg**. Il consiste à faire une moyenne pondérée des poids des clients :

$$w_{global} = \sum_{i=1}^K \frac{n_i}{\sum_{j=1}^K n_j} w_i$$

où w_i représente les poids du modèle local du client i , et n_i le nombre d'échantillons de ce client.

3 Implémentation from scratch

3.1 Baseline centralisée

La première étape a été d'entraîner un modèle centralisé sur l'ensemble de MNIST afin d'obtenir une référence. Le modèle utilisé était un MLP simple composé de couches entièrement connectées.

Le modèle centralisé atteint environ **97%** d'accuracy après deux époques. Cette valeur sert de baseline pour comparer les résultats obtenus en Federated Learning.

3.2 Première version fédérée

Ensuite, le dataset a été divisé entre plusieurs clients. Dans un premier scénario IID, chaque client reçoit une portion homogène du dataset.

Après 3 rounds de Federated Learning, le modèle fédéré atteint environ **94.88%**. Le résultat est légèrement inférieur à la baseline centralisée, ce qui est attendu, mais il reste très correct.

TABLE 1 – Résultats du Sprint 1

Expérience	Accuracy
Centralisé MLP	0.9700
Federated Learning IID, 3 rounds	0.9488

Cette première étape a permis de valider que l'implémentation de FedAvg fonctionnait correctement.

4 Étude des distributions IID et non-IID

Le Sprint 2 s'est concentré sur des scénarios plus réalistes, notamment les données non-IID.

Trois distributions ont été testées :

- **IID** : chaque client reçoit une distribution similaire des classes ;
- **non-IID par classes** : chaque client ne possède qu'un sous-ensemble de classes ;
- **non-IID déséquilibré** : les clients ont des quantités de données différentes.

Le cas non-IID par classes est le plus difficile, car chaque client ne voit qu'une partie du problème. Par exemple, un client peut ne voir que les chiffres 0 et 1, tandis qu'un autre ne voit que les chiffres 2 et 3.

Le principal défi observé est que le non-IID par classes dégrade fortement les performances, car les gradients locaux des clients peuvent aller dans des directions différentes.

5 Comparaison des modèles

5.1 MLP

Le MLP a été utilisé comme modèle de départ. Il donne de bons résultats en IID, mais ses performances chutent en non-IID par classes.

Cette limite est logique : le MLP traite l'image comme un vecteur de pixels et exploite moins bien la structure spatiale de l'image.

5.2 CNN V1

Pour améliorer les résultats, un CNN a été introduit. Contrairement au MLP, le CNN utilise des convolutions et peut capturer des motifs locaux dans les images.

CNN V1 améliore nettement les performances, surtout en non-IID par classes.

5.3 CNN V2

CNN V2 ajoute Batch Normalization et Dropout. En apprentissage centralisé, BatchNorm est souvent utile, mais en Federated Learning non-IID, elle peut poser problème.

Chaque client calcule ses propres statistiques locales. Comme les distributions des clients sont différentes, ces statistiques ne sont pas forcément compatibles lors de l'agrégation. Résultat : CNN V2 s'effondre en non-IID par classes.

5.4 CNN V3

CNN V3 supprime BatchNorm tout en conservant Dropout. Il est donc plus adapté au contexte distribué. Même si CNN V1 reste très performant, CNN V3 est retenu comme compromis plus robuste, car il évite une technique problématique en Federated Learning.

TABLE 2 – Comparaison des modèles

Modèle	IID	Non-IID classes	Non-IID imbalanced
MLP	~ 0.95	0.7328	0.9720
CNN V1	~ 0.987	~ 0.85	~ 0.986
CNN V2	~ 0.985	~ 0.11	~ 0.988
CNN V3	~ 0.985	~ 0.83	~ 0.989

Ces résultats montrent que le choix du modèle est important, mais aussi que certaines techniques classiques du deep learning ne sont pas toujours adaptées au Federated Learning.

6 Exploration avec autoencodeur

Une approche avec autoencodeur a aussi été testée. L'idée était de compresser les images dans un espace latent plus compact, puis d'entraîner un classifieur sur ces représentations.

L'autoencodeur apprend à reconstruire les images, puis son encodeur transforme chaque image en vecteur latent. Ensuite, un MLP est utilisé pour classifier ces vecteurs.

Les résultats montrent que cette approche reste correcte en IID, mais n'améliore pas le cas non-IID par classes. La compression entraîne probablement une perte d'informations discriminantes importantes.

L'autoencodeur était une piste intéressante, mais dans cette configuration, il n'a pas permis d'améliorer la robustesse au non-IID.

7 Simulation distribuée avec Streamlit

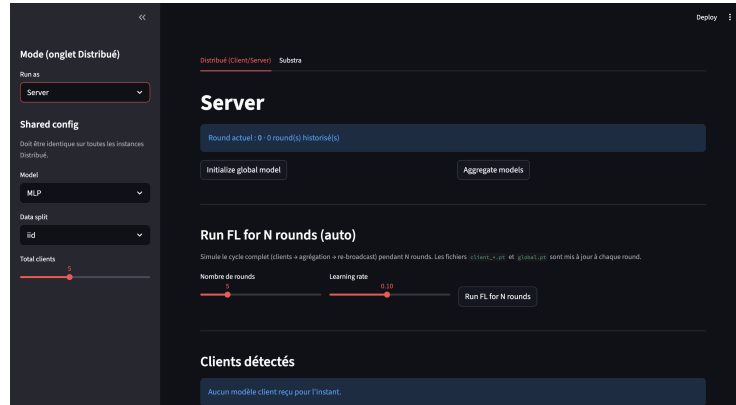


FIGURE 1 – Interface streamlit coté Serveur

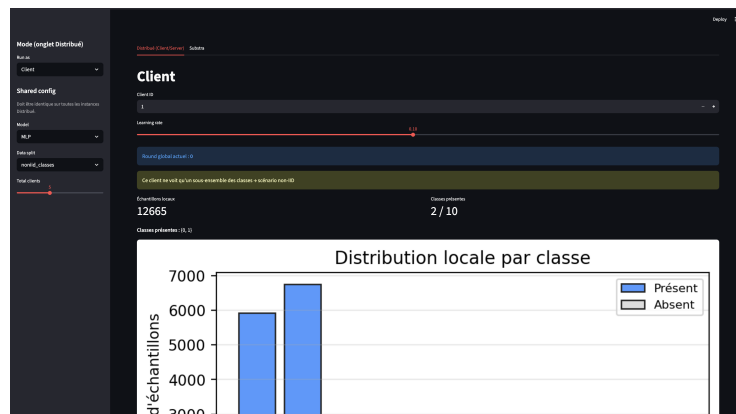


FIGURE 2 – Interface streamlit coté Client

Pour rendre le projet plus concret, une application Streamlit a été développée. L'objectif était de simuler plusieurs machines : des clients et un serveur central.

L'application possède deux modes :

- **Client** : charge le modèle global, entraîne localement, sauvegarde son modèle ;
- **Server** : récupère les modèles clients, applique FedAvg et évalue le modèle global.

Plusieurs instances Streamlit peuvent être lancées sur des ports différents :

```
streamlit run app.py --server.port 8501 # Server
streamlit run app.py --server.port 8502 # Client 1
streamlit run app.py --server.port 8503 # Client 2
streamlit run app.py --server.port 8504 # Client 3
```

Les modèles sont échangés via des fichiers .pt. Par exemple :

- models/global.pt
- models/client_1.pt
- models/client_2.pt

L'interface permet aussi d'afficher ce que voit chaque client : classes présentes, nombre d'échantillons et histogramme de distribution locale. Cela rend le non-IID beaucoup plus facile à comprendre visuellement.

8 Intégration Substra

Substra a été ajouté pour comparer l'implémentation from scratch avec un framework spécialisé dans le Federated Learning.

La différence principale est que l'implémentation from scratch donne un contrôle total sur chaque étape, tandis que Substra gère une partie de l'orchestration : clients, tâches, rounds et agrégation.

TABLE 3 – From scratch vs Substra

Aspect	From scratch	Substra
Contrôle	Total	Partiel
Objectif	Comprendre	Valider dans un cadre réaliste
Orchestration	Manuelle	Gérée par le framework
Évaluation	Global test set	Évaluation locale par organisation

Afin d'analyser le comportement des modèles en contexte homogène, nous comparons ici l'évolution de l'accuracy globale pour le MLP et le CNN V3 dans un scénario IID.

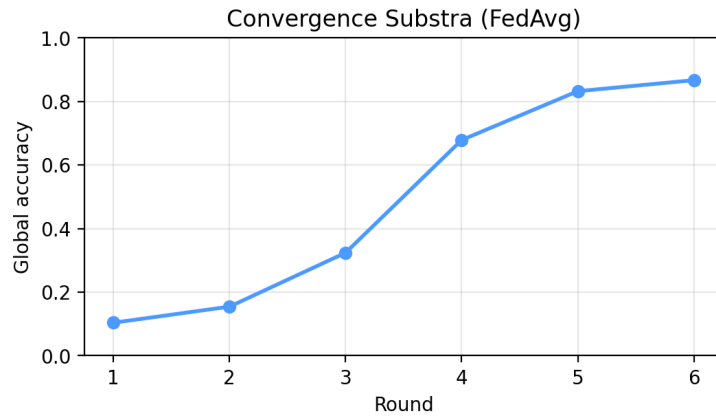


FIGURE 3 – Courbe Accuracy MLP - iid

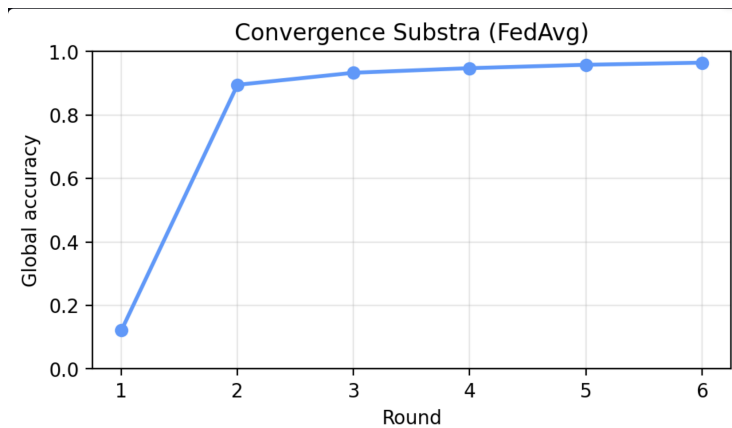


FIGURE 4 – Courbe Accuracy CNN v3 - iid

Les figures présentent la convergence des deux modèles au fil des rounds de Federated Learning. On observe que les deux architectures convergent de manière stable, confirmant que FedAvg fonctionne

efficacement lorsque les données sont distribuées de manière homogène.

Cependant, le CNN V3 atteint une accuracy plus élevée que le MLP, ce qui s'explique par sa capacité à exploiter les structures spatiales des images. Le MLP, en revanche, traite les images comme de simples vecteurs, ce qui limite ses performances.

Ces résultats confirment que, même en contexte IID, le choix de l'architecture reste un facteur déterminant pour la performance du modèle global.

Un point important a été observé : en contexte non-IID, Substra évalue par défaut sur les données locales d'une organisation, et non sur un test set global centralisé. Ce comportement est logique dans un cadre réel, car les données ne doivent pas être centralisées.

Cependant, pour un projet académique, cela peut rendre l'interprétation plus difficile. Par exemple, si le test est effectué sur un client qui ne possède que deux classes, l'accuracy peut être faible, autour de 20%, même si cela ne reflète pas directement la performance globale du modèle.

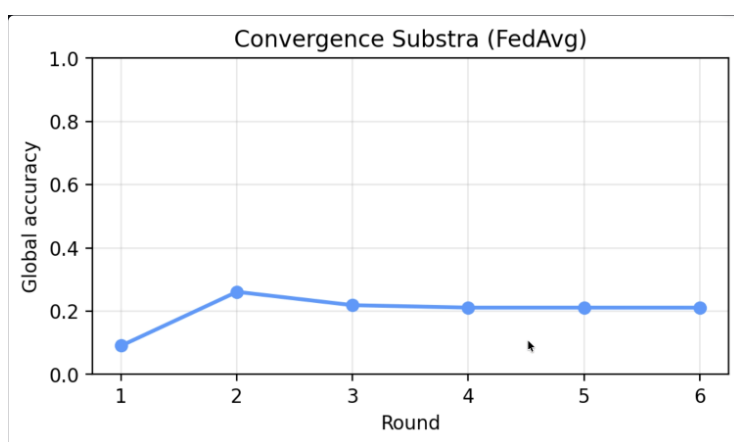


FIGURE 5 – Courbe Accuracy CNN v3 - non-idd classes

Substra montre bien la différence entre un benchmark académique, où l'on veut souvent une accuracy globale, et un vrai système fédéré, où chaque organisation garde ses données et évalue localement.

9 Discussion

Le projet montre plusieurs résultats importants.

Premièrement, FedAvg fonctionne très bien lorsque les données sont IID. Les résultats sont proches du modèle centralisé.

Deuxièmement, le non-IID par classes reste le scénario le plus difficile. Chaque client ne voit qu'une partie des classes, ce qui rend l'agrégation moins stable.

Troisièmement, le choix du modèle a un impact important. Le passage du MLP au CNN améliore nettement les performances sur MNIST. Cependant, CNN V2 montre que toutes les techniques classiques ne sont pas forcément adaptées au Federated Learning.

Enfin, Streamlit et Substra ont permis d'aller au-delà du simple notebook. Streamlit a rendu le système plus visuel et interactif, tandis que Substra a permis de comprendre les contraintes d'un framework plus proche du réel.

10 Limites et perspectives

La première limite est que MNIST reste un dataset simple. Il est utile pour comprendre les mécanismes, mais il ne représente pas la complexité de données réelles.

Une perspective naturelle serait de tester le projet sur des datasets plus complexes, comme Fashion-MNIST ou CIFAR-10.

Une autre perspective serait d'utiliser MIMIC-IV pour un cas médical. Contrairement à MNIST, MIMIC-IV est tabulaire et ne contient pas d'images. Il faudrait donc utiliser un MLP plutôt qu'un CNN, et définir une tâche comme la prédiction de mortalité ou de risque patient.

Enfin, une amélioration possible serait d'implémenter FedProx, une variante de FedAvg plus robuste en contexte non-IID.

11 Conclusion

Ce projet a permis d'étudier le Federated Learning de manière progressive.

L'implémentation from scratch a permis de comprendre les mécanismes internes : entraînement local, agrégation FedAvg, rounds et impact de la distribution des données.

Les expériences ont montré que le non-IID par classes est le principal défi. Le MLP donne une bonne baseline, mais les CNN sont plus adaptés à la classification d'images. L'expérience avec BatchNorm a aussi montré qu'une technique efficace en centralisé peut devenir problématique en contexte distribué.

L'interface Streamlit a permis de simuler plusieurs clients et un serveur, rendant le projet plus concret. L'intégration de Substra a ensuite apporté une vision plus réaliste des contraintes du Federated Learning, notamment sur l'évaluation locale.

Le principal enseignement du projet est que le Federated Learning ne dépend pas seulement du modèle, mais aussi de la distribution des données, du protocole d'agrégation et de la manière dont les performances sont évaluées.